

ACME Corp

Estrategia de calidad

ESTADO

Propuesta

ROL

QA Leader

HORIZONTE

90 días

Reto	Resolución QA Leader de Alegra
Audiencia	Dirección de Ingeniería y Producto
Contexto	60 desarrolladores y 9 equipos
Alcance	Estrategia, IA y criterio técnico

1 Propuesta para los primeros 90 días

Propongo empezar por los riesgos que más afectan la confianza del usuario: mensajes que se pierden, se duplican, muestran información distinta o llegan a quien no corresponde. El piloto se concentrará en envío y edición de mensajes, y servirá para probar una forma de trabajo que luego puedan adoptar otros equipos.

Con 60 desarrolladores, 9 equipos y dos personas de QA, la validación debe formar parte del trabajo de cada equipo. QA acompañará las decisiones de mayor riesgo y ayudará a crear pruebas que los equipos puedan mantener.

Decisión	Aplicación concreta	Beneficio esperado
Empezar con un piloto	Iniciar con el equipo de envío de mensajes. Si otro equipo guarda o sincroniza los mensajes, incorporarlo para validar las integraciones.	Concentrar el esfuerzo en un recorrido crítico.
Prevenir desde el refinamiento	Producto, Desarrollo y QA definen situaciones de uso, error y límite, y qué debe ocurrir en cada una, antes de implementar.	Resolver dudas antes de que se conviertan en defectos.
Proteger cada entrega	Automatizar permisos, reintentos y consistencia; comprobar también el recorrido desde la interfaz.	Detectar regresiones con rapidez.
Apoyar el mantenimiento con IA	Usar el prototipo para comparar pares de casos con revisión humana. Evaluar su ampliación a una muestra de hasta 200 casos.	Reducir trabajo repetido sin perder pruebas útiles.
Ampliar según resultados	Incorporar otro equipo cuando el piloto tenga controles confiables y responsables de mantenerlos.	Crece sin sobrecargar a QA ni a los equipos.

Qué espero lograr

Al día 90, el piloto tendrá 20 escenarios críticos automatizados, responsables definidos y un tablero que permita comparar resultados con el inicio. La meta será reducir un 20 % relativo la proporción de despliegues que causan fallos, siempre que haya suficientes despliegues para evaluar la tendencia.

Apoyo de Ingeniería

Necesito el respaldo de un líder de Ingeniería, tiempo reservado en los sprints y acceso a incidentes, repositorios y procesos de despliegue. Empezaré con el personal y las herramientas disponibles. Cualquier inversión adicional se justificará con las necesidades y los resultados del piloto.

2 Prioridades y responsabilidades

Prioridad	Por qué atenderla	Primera acción
Permisos y privacidad	Un usuario no debe acceder a mensajes de otra empresa o conversación.	Probar accesos permitidos y rechazados desde la API.
Pérdida y duplicación	Afectan directamente el servicio de mensajería.	Probar guardado, reintentos y caídas de conexión.
Ediciones inconsistentes	Dos usuarios pueden tomar decisiones sobre textos diferentes.	Probar versiones, cambios simultáneos y reconexión.
Regresiones frecuentes	Un cambio puede romper funciones que ya servían.	Ejecutar pruebas críticas antes de desplegar.
Casos costosos de mantener	Consumir tiempo en casos repetidos deja menos espacio para riesgos nuevos.	Depurar una muestra con apoyo de IA.
Crecimiento de tráfico	El reto plantea millones de mensajes al día.	Medir capacidad y localizar el primer límite.

Confirmaré este orden con los incidentes recientes, los usuarios afectados y la frecuencia de cambios. La exposición de información y la pérdida de datos tendrán prioridad aunque sean poco frecuentes.

Quién hace qué

Producto define las reglas y los resultados esperados. Desarrollo implementa y mantiene las pruebas de sus servicios. Como QA Lead, priorizaré los riesgos, acordaré los controles y revisaré los resultados. La segunda persona de QA trabaja en los casos críticos, la exploración y la automatización, según su experiencia. Plataforma apoya ambientes y despliegues cuando exista ese equipo.

Cada equipo tendrá un referente con tiempo reservado para coordinar estas tareas. El responsable técnico decide las liberaciones con base en los controles acordados; Ingeniería resuelve excepciones de riesgo junto con Producto y QA.

Cómo abordar la resistencia

Empezaré con un fallo real que el equipo quiera evitar. Construiremos juntos el primer control y comprobaremos que sus alertas sean útiles antes de hacerlo obligatorio. Los testers manuales participarán desde el diseño de escenarios y tendrán acompañamiento práctico en API y automatización.

Durante los 90 días dejaré fuera la automatización de toda la suite, una migración general de herramientas y la predicción de incidentes con IA. Primero necesito controles básicos confiables y datos para decidir qué ampliar.

3 QA durante todo el ciclo de software

Shift left significa involucrar a QA desde que se define la necesidad. Así podemos detectar reglas incompletas antes de escribir código. El trabajo continúa durante el desarrollo, la entrega y el análisis de lo que ocurre en producción.

Momento	Participación de QA y del equipo
Definir y refinar	Acordar casos de uso, errores, permisos y límites. Resolver dudas que cambien el comportamiento esperado.
Diseñar y planificar	Revisar integraciones y fallos posibles. Incluir pruebas, datos y ambientes en la estimación de la historia.
Desarrollar	Revisar casos con Desarrollo y automatizar las reglas cerca del código. Actualizar las pruebas si cambia el requisito.
Integrar y desplegar	Comprobar los controles y la forma de recuperar el servicio si falla la entrega.
Operar y mejorar	Relacionar incidentes con los controles que faltaron. Llevar una mejora concreta al siguiente sprint.

Ejemplo de refinamiento

Historia: como usuario, quiero enviar un mensaje y saber si fue enviado, incluso cuando mi conexión sea inestable. Producto aclara qué significa cada estado; Desarrollo propone cómo guardar y reconocer la operación; QA plantea situaciones que podrían romper ese comportamiento.

```
Dado que el servidor ya guardó el mensaje
Y la respuesta no llegó al cliente
Cuando el cliente reintenta la misma operación
Entonces recibe el mismo identificador
Y el destinatario ve una sola copia
```

También acordamos que un usuario sin permiso no puede enviar, que una clave de operación usada con otro texto se rechaza y que “enviado”, “entregado” y “leído” requieren confirmaciones diferentes. Estos ejemplos se convierten en criterios de aceptación y pruebas.

Cuándo está terminada una historia

Una historia termina cuando cumple sus criterios, tiene revisión de código, pasan las pruebas relevantes y queda evidencia vinculada al cambio. Si afecta una integración o el despliegue, también debe comprobar compatibilidad y recuperación. No se declara terminada para dejar sus pruebas pendientes en el siguiente sprint.

4 Plan de trabajo de 90 días

Etapa	Trabajo principal	Resultado para avanzar
Días 1 a 30	Revisar hasta 10 incidentes recientes, conocer responsables y herramientas, elegir el piloto y medir la situación inicial. Empezar por envío, reintento, permisos, edición y conflicto de versiones.	10 escenarios críticos automatizados. Cada prueba tiene responsable y se ha comprobado su estabilidad.
Días 31 a 60	Completar los 16 escenarios funcionales P0 y los 4 controles transversales. Evaluar el asistente con 60 pares; si cumple las metas, ampliar la revisión a hasta 200 casos. Ejecutar carga y recuperación.	20 escenarios críticos ejecutados antes de liberar. Evidencia vinculada a historias y primera evaluación de IA y capacidad.
Días 61 a 90	Corregir pruebas inestables, formar a los responsables y comparar las métricas. Documentar ejemplos reutilizables. Incluir un tercer equipo solo si el piloto se sostiene.	El equipo mantiene sus controles. Se presentan resultados, límites encontrados y prioridades de la siguiente etapa.

Distribución del trabajo

El QA Lead se concentra en prioridades, refinamiento, coordinación y medición. La segunda persona trabaja con Desarrollo en escenarios, datos y pruebas. Reservaré alrededor del 20 % de la capacidad conjunta para incidentes y ajustes; revisaré la distribución cada semana.

Como punto de partida, acordaría un día de Desarrollo por semana en cada equipo piloto durante la construcción de los controles, y unas dos horas semanales de su referente. Este tiempo debe reflejarse en el sprint. Las pruebas son parte del cambio, no una actividad adicional que se asume sin estimación.

Si el plan encuentra obstáculos

Si no hay apoyo suficiente de Desarrollo, empezaré con un solo equipo y los 10 escenarios de mayor riesgo. Si falta un ambiente estable, resolveremos primero los datos y dependencias necesarias. Si la IA no aporta resultados confiables, mantendremos la revisión manual. En cada caso informaré qué se posterga y qué riesgo queda pendiente.

Haré una revisión quincenal con Ingeniería y Producto para decidir ajustes. Antes de exigir que una prueba bloquee entregas, pediré al menos 20 ejecuciones consecutivas estables, con datos representativos y un responsable de corregirla.

5 Cómo medir resultados y ampliar el modelo

Durante las primeras dos semanas estableceré la línea base con datos disponibles. Compararé periodos de duración similar y mostraré los conteos junto con los porcentajes. Las siguientes son metas iniciales; se ajustan si la muestra o las condiciones no permiten una comparación válida.

Indicador	Cómo se obtiene	Meta inicial
Cambios que causan fallos	Despliegues que requieren corrección, reversión o intervención por un fallo / despliegues totales. Datos de CI/CD e incidentes.	Reducir 20 % relativo al día 90.
Protección de riesgos críticos	Escenarios críticos implementados, ejecutados y aprobados sobre el candidato a liberar.	20 de 20 antes de liberar.
Relación con requisitos	Historias del piloto con criterios y evidencia de pruebas / historias terminadas.	Al menos 90 %.
Tiempo de validación	Duración del pipeline. p95 es el tiempo que no supera el 95 % de las ejecuciones.	PR p95 $\leq$ 10 min; candidato $\leq$ 15 min.
Pruebas inestables	Ejecuciones con fallos confirmados de la prueba o ambiente / ejecuciones totales.	$\leq$ 2 %; ningún riesgo crítico sin control.
Efecto sobre la entrega	Mediana del tiempo entre cambio de código y producción; tiempo de recuperación tras fallos.	Investigar si la entrega empeora más de 15 % durante dos semanas.

Ejemplo: pasar de 8 fallos en 40 despliegues (20 %) a 6 en 40 (15 %) sería una reducción relativa del 25 %. Es una forma de calcular el resultado, no un dato de ACME. QA consolida el tablero; los dueños de servicios validan incidentes y despliegues.

Con 100 y 150 desarrolladores

Con 100 desarrolladores, organizaría referentes por dominio y evaluaría sumar una persona que ayude a los equipos a implementar controles. Con 150, evaluaría un núcleo de 4 a 5 personas, incluyendo liderazgo y apoyo especializado en rendimiento, confiabilidad o seguridad. Aprovecharía las capacidades de Plataforma y Seguridad que ya existan.

Estas cifras son escenarios de planificación. La decisión dependerá de la demanda: solicitudes que esperan más de una semana, riesgos críticos sin responsable o incidentes repetidos por falta de especialización. Mantendría comunes los criterios de entrega y las métricas; cada equipo conservaría la implementación de sus pruebas.

6 Pruebas y controles antes de liberar

Automatizaré reglas estables y repetitivas. Las pruebas unitarias comprueban pequeñas partes del código; las de API e integración verifican permisos, datos y comunicación entre servicios. Las pruebas de contrato revisan compatibilidad. Las E2E recorren la aplicación como un usuario y se limitarán a envío y edición esenciales.

Momento	Controles obligatorios según el cambio
Cada solicitud de cambio o PR	Compilación, análisis estático, pruebas unitarias, detección de secretos y dependencias vulnerables. Añadir API, integración, permisos y contratos de las partes afectadas.
Candidato a producción	Ejecutar los 16 escenarios P0 y X01 a X04 sobre el mismo artefacto que se desplegará. Revisar resultados y defectos abiertos.
Después de desplegar	Comprobar envío y edición con cuentas de prueba, observar errores y confirmar que el servicio puede recuperarse.
Ejecución programada	Ampliar regresión y carga. Antes de liberar cambios de rendimiento o migraciones riesgosas, ejecutar también las pruebas específicas.

CI/CD es el proceso automático que integra cambios y prepara o ejecuta despliegues. Los objetivos de tiempo ayudan a detectar lentitud; si un control necesario tarda más, se optimiza el proceso sin omitirlo. La pérdida de datos, el acceso indebido y los fallos graves del recorrido central bloquean la entrega.

Mantenimiento y pruebas manuales

Cada caso tendrá ID, requisito o riesgo, datos, resultado esperado, responsable y vínculo a su ejecución. El cambio de código debe actualizar las pruebas afectadas. Un incidente importante añadirá o mejorará un control que permita detectar su repetición.

Revisaré los casos antiguos para conservar los útiles, fusionar duplicados con aprobación y retirar los obsoletos con historial. Las pruebas inestables tendrán responsable y plazo de corrección; un riesgo crítico necesitará una verificación equivalente mientras se corrige su automatización.

Mantendré exploración manual de accesibilidad, claridad de errores, conexión inestable e interacción entre dispositivos. Usaré sesiones breves con un objetivo y evidencia. Si faltan herramientas adecuadas, consideraría Playwright para interfaz y k6 para carga; primero evaluaría las existentes.

7 Producción y millones de mensajes diarios

Además de detectar defectos antes de liberar, necesito saber qué ocurre después. Cada mensaje de prueba tendrá un identificador para seguir su aceptación, almacenamiento y entrega, sin registrar conversaciones reales ni secretos.

Objetivos iniciales del servicio

Propondría que el 99,9 % de los intentos válidos de envío se guarden correctamente, medido en 28 días. Como referencia inicial, p95 de aceptación ≤ 500 ms y entrega a usuarios conectados ≤ 2 s. Confirmaría estos valores con Producto y una medición real; el tiempo de un usuario desconectado se evalúa desde su reconexión. Cualquier pérdida confirmada o acceso indebido requiere investigación inmediata.

Liberación y recuperación

Si la plataforma lo permite, desplegaré primero a una fracción pequeña del tráfico. Compararé errores, latencia y acumulación de mensajes pendientes con la versión anterior. Ante un deterioro sostenido, detendré la ampliación y aplicaré la recuperación acordada. Una migración de datos requiere comprobar si es seguro revertir o si hace falta una corrección hacia adelante.

El responsable técnico coordina el incidente. Después revisamos la causa y añadimos una mejora verificable al siguiente sprint, sin centrar el análisis en culpas individuales.

Modelo de carga a seis meses

Usaré 5 millones de envíos diarios como escenario de planificación: aproximadamente 58 por segundo en promedio. Un pico de 10 veces ese promedio sería 580 envíos/s. Con 3 destinatarios por mensaje, serían unas 1.740 entregas/s. Las conexiones simultáneas se medirán aparte; estos supuestos deben ajustarse al uso real.

Prueba	Perfil inicial y qué comprobar
Carga esperada	580 envíos/s más 64 ediciones/s durante 30 min. Revisar latencia, errores, datos y crecimiento de colas.
Pico	Duplicar los envíos durante 5 min. Comprobar recuperación sin pérdida y sin reintentos descontrolados.
Duración	4 horas al 60–70 % de la capacidad sostenible medida. Buscar fugas de memoria y acumulación progresiva.
Límite y fallos	Subir carga gradualmente y detener un consumidor de prueba. Identificar el límite y comprobar recuperación y aislamiento entre empresas.

El generador enviará al ritmo fijado, sin esperar cada respuesta. Reportaré operaciones no generadas por falta de capacidad del propio generador. Además de tiempos, compararé IDs y versiones aceptadas contra almacenamiento y clientes; una respuesta exitosa por sí sola no demuestra integridad.

8 Prototipo de IA para revisar casos

Como QA Lead, propongo usar IA para reducir el trabajo de revisar casos repetidos y aclarar su relación con los requisitos. Para demostrar la propuesta, presento QA Lab: un prototipo web funcional con revisión humana.

Acceso público: [Abrir QA Lab](#)

El prototipo compara dos casos por consulta y recibe hasta cinco requisitos. Se puede empezar con tres ejemplos ficticios, editar el formulario o cargar un archivo JSON. Las recomendaciones se generan al consultar OpenAI; no son respuestas precargadas.

Paso	Qué hace el prototipo
1. Preparar	Recibe identificadores, condiciones, pasos, resultados esperados y riesgos. Comprueba los datos obligatorios y los límites de entrada.
2. Analizar	Compara el par y sugiere duplicado, casos distintos o revisión necesaria. Explica la decisión y cita los datos que la respaldan.
3. Validar	Comprueba el formato, los IDs y las citas. Si la respuesta no es válida, reintenta una vez; si vuelve a fallar, muestra un error sin recomendar cambios.
4. Decidir	QA revisa diferencias y requisitos, añade una nota y acepta o rechaza la sugerencia. Aceptar no fusiona ni elimina casos.
5. Conservar evidencia	Descarga un JSON con la entrada exacta, la respuesta, los datos de ejecución y el estado de revisión humana.

Cómo lo incorporaría al trabajo de QA

En refinamiento, lo usaría para identificar reglas incompletas y llevar preguntas a Producto antes del desarrollo. Durante el mantenimiento, lo usaría para comparar casos similares sin perder variantes de permisos, conexión o resultados.

Alcance de esta versión

La selección de pares es manual. La revisión de una suite de hasta 200 casos sigue siendo una ampliación propuesta; aún no hay selección automática de candidatos, importación CSV, integración con un gestor de pruebas ni historial compartido.

8.1 Controles y trazabilidad del prototipo

El navegador envía los casos al servidor. El servidor consulta OpenAI Responses API, valida la respuesta y devuelve la recomendación. La clave permanece en el servidor. La versión publicada usa gpt-4.1-mini-2025-04-14 y el prompt qa-review-v2.1.

Instrucciones y revisión de la respuesta

El modelo recibe la instrucción de tratar los casos como datos, no como órdenes, y de no inventar reglas ni identificadores. Si las condiciones y los pasos son iguales pero los resultados se contradicen, debe pedir revisión de la regla. Las citas deben coincidir con el texto recibido.

El servidor exige evidencia de ambos casos, IDs válidos y revisión humana. Si el modelo propone consolidar casos con funcionalidades o etiquetas de riesgo diferentes, la recomendación pasa a revisión manual. Estos controles ayudan a detectar errores, pero no garantizan que la interpretación del modelo sea correcta.

Datos y límites de uso

Solo deben usarse datos ficticios o revisados. Hay detección básica de correos y patrones de credenciales; no es anonimización completa. El contenido se envía a OpenAI para analizarlo. Se configura store: false, sin asumir que eso elimina toda retención del proveedor.

Se aceptan entradas de hasta 24 KB. La aplicación limita el uso global a 100 llamadas al proveedor por día UTC y 1.000 en total, incluidos reintentos e intentos fallidos. Estos contadores no equivalen a un límite de gasto en dólares.

Qué evidencia queda disponible

La descarga incluye los casos y requisitos enviados, la recomendación, las citas, la decisión humana y su nota. También incluye fecha, modelo, versión del prompt, ID de respuesta, duración, tokens y una huella SHA-256 para identificar la entrada.

La aplicación no guarda los casos ni las revisiones en una base de datos; solo conserva contadores de uso. La revisión queda en la sesión de la página y debe descargarse para conservarla. Los casos originales no se modifican.

Si la IA falla

Un error de conexión, saldo, cuota o validación se informa sin reemplazarlo por una respuesta simulada. Como QA Lead, mantendría la revisión manual disponible y no haría depender una entrega de este asistente.

9 Pruebas realizadas y evaluación de la IA

El 17 de septiembre de 2026 se probaron tres ejemplos con IA real. Con el prompt qa-review-v2.1, los tres coincidieron con la decisión esperada y pasaron la validación en el primer intento. Son pruebas de funcionamiento sobre ejemplos conocidos; no demuestran precisión en producción.

Prueba	Resultado observado	Qué comprueba
Mismas condiciones, pasos y resultado	DUPLICATE	Reconoce el par duplicado.
Miembro autorizado frente a no miembro	DISTINCT	Conserva la diferencia de permisos.
Misma edición y resultados opuestos, sin plazo definido	NEEDS_REVIEW	Pide aclarar la regla.
Variación enviada al servidor: uno frente a dos mensajes	DISTINCT · HTTP 200	Procesa datos modificados y devuelve la entrada exacta junto con el resultado.

Además, pasaron 17 pruebas automáticas de contrato y validación. Comprueban, entre otros controles, IDs inventados, citas incorrectas, campos obligatorios y revisión humana. No son 17 evaluaciones de precisión del modelo.

La primera prueba del prompt anterior mostró dos citas rechazadas y una decisión distinta de la esperada. Se aclararon las instrucciones y se repitieron los tres ejemplos. Este hallazgo refuerza la necesidad de validar las salidas y conservar la revisión humana.

Metas para decidir si lo ampliaría

Prepararé 60 pares revisados por QA y el referente: 20 duplicados, 20 distintos y 20 ambiguos. Separaré 30 para ajustar y 30 para evaluar sin nuevos ajustes, sin repetir casos entre grupos. Reportaré conteos y errores además de porcentajes.

Medida	Meta del piloto aún no demostrada
Duplicados	≥95 % de sugerencias correctas y detección de ≥70 % de duplicados reales.
Trazabilidad y cobertura	≥90 % de vínculos correctos. Ninguna consolidación aprobada debe perder una diferencia relevante.
Tiempo neto	Reducir ≥25 % la mediana en 20 comparaciones equivalentes, incluyendo preparación, revisión y corrección.

Propondría un presupuesto de USD 50 para el piloto, con seguimiento de consumo y un mecanismo de corte antes de ampliar su uso. Ese corte monetario aún no está implementado. La demostración actual no acredita ahorro de tiempo ni las metas de calidad; un cambio de modelo o prompt exige reevaluar.

10 Reglas para probar envío y edición

Elijo envío por ser el recorrido central y edición porque permite evaluar permisos, cambios simultáneos y sincronización. Para que los resultados esperados sean claros, propongo las reglas siguientes. Los plazos y límites son decisiones de diseño para resolver el ejercicio; se confirmarían antes de implementarlos.

Tema	Regla propuesta
Permisos	La empresa y el usuario se obtienen de la sesión autenticada. Enviar requiere pertenecer a la conversación. Editar requiere además ser el autor; el servidor valida cada operación.
Confirmaciones	“Enviado” significa guardado de forma durable; “entregado” requiere confirmación de un dispositivo destinatario; “leído” requiere un evento de lectura.
Reintentos	Idempotencia significa que repetir la misma operación no la aplica dos veces. Cada operación lleva una clave válida durante 24 h. Misma clave y datos devuelve el mismo resultado; datos distintos producen conflicto.
Reintento tardío	Al vencer la clave, el cliente consulta el estado antes de reenviar. Si no puede comprobarlo, muestra la situación como pendiente de resolver.
Texto y orden	Entre 1 y 4.000 puntos de código Unicode, normalizados en NFC, no solo espacios. Sin ejecución de HTML o scripts. El servidor ordena por conversación; no usa el reloj del cliente.
Edición	Solo antes de 15 min desde la creación; en el límite exacto se rechaza. La operación indica la versión esperada. Si otro cambio se adelantó, se informa conflicto.
Reintento de edición	Tras validar permisos, una operación ya completada devuelve su resultado sin crear otra versión, incluso si al reintentar ya pasó el plazo de edición.
Estado final	Editar conserva ID, fecha de creación, orden y recibos. El mismo texto no crea versión. Un borrado confirmado prevalece; un evento viejo no restaura contenido.

Se muestra una marca de editado y se registra autor, momento y versión con acceso restringido. La política de historial visible y los límites de notificaciones externas se acuerdan con Producto.

En las matrices, P0 protege riesgos críticos para liberar; P1 cubre límites e integraciones importantes; P2 amplía la exploración. Una prueba P1 también es obligatoria cuando el cambio afecta directamente su regla.

11 Envío de mensajes y riesgos críticos

Caso	Situación y resultado esperado	Validación
S01	Enviar entre miembros autorizados. Debe quedar un registro guardado y una copia visible, con estados de envío, entrega y lectura correctamente diferenciados.	Integración y E2E.
S02	Perder la respuesta después del guardado y reintentar dos veces a la vez. Debe devolver el mismo ID sin duplicar el mensaje visible.	Integración con fallo de red.
S03	Intentar enviar como no miembro, usuario de otra empresa o miembro revocado. Debe rechazar sin guardar, emitir eventos ni exponer información ajena.	API de permisos.
S04	Reutilizar la clave con otro texto. Debe informar conflicto y conservar el mensaje original sin crear otro.	API e integración.
S05	Enviar sin conexión y reconectar varias veces. El texto pendiente se conserva y termina en una sola copia al recuperarse.	Integración y exploración.
S06	Recibir confirmaciones repetidas o fuera de orden. Los estados no retroceden ni muestran lectura o entrega sin evidencia.	Integración de eventos.
S07	Fallar antes de guardar y después de guardar, antes de publicar. El primer caso no confirma envío; el segundo recupera la entrega sin pérdida ni copia adicional.	Integración con fallos controlados.
S08	Enviar scripts y etiquetas HTML. Deben tratarse como texto seguro en los clientes soportados, sin ejecutar código ni romper la vista.	API e interfaz.

Estos ocho escenarios se ejecutan sobre cada candidato a liberar. En cada PR se seleccionan además los afectados por el cambio. La verificación compara respuesta, almacenamiento, eventos y vista del receptor.

12 Envío de mensajes y casos adicionales

Caso	Situación y resultado esperado	Validación
S09 P1	Probar vacío, espacios, 1, 3.999, 4.000 y 4.001 puntos de código, emojis y texto normalizado. Cliente y servidor aplican el mismo límite sin truncar.	Unidad y API; idiomas manual.
S10 P1	Enviar desde dos usuarios a la vez con relojes distintos. Ambos clientes deben ver el orden definido por el servidor.	Integración.
S11 P1	Vencer la sesión antes del envío o después de perder su respuesta. Debe exigir autorización; renovar la sesión no duplica lo ya enviado.	API e integración.
S12 P1	Generar una ráfaga desde una empresa. Debe aplicar el límite acordado sin saturar a otras ni provocar reintentos continuos.	API y carga.
S13 P1	Reintentar antes, en y después de 24 h. Antes se reconoce la operación; al vencer, se consulta el estado antes de reenviar.	Integración con reloj controlado.
S14 P1	Usar una versión anterior aún soportada. Respuestas y eventos nuevos no deben romper el envío ni sus estados.	Contratos.
S15 P2	Usar teclado, lector de pantalla y red lenta. El estado y los errores deben entenderse; el reintento debe ser accesible.	Exploración y accesibilidad.
S16 P1	Enviar a un grupo con varios dispositivos y un usuario que salió. Validar destinatarios y recibos según la política de membresía acordada.	Integración.

Automatizaré reglas y resultados estables. La claridad de mensajes, la accesibilidad asistida y el uso entre plataformas se revisan también de forma manual, con objetivo y evidencia registrados.

13 Edición de mensajes y riesgos críticos

Caso	Situación y resultado esperado	Validación
E01	El autor edita dentro del plazo con la versión vigente. Dos clientes muestran el nuevo texto y la marca de editado; se conserva ID, orden, creación y recibos.	API, integración y E2E.
E02	Intentar editar como no autor, usuario de otra empresa o miembro revocado. Debe rechazar sin cambiar ni revelar contenido.	API de permisos.
E03	Editar la misma versión desde dos dispositivos a la vez. Una edición confirma y la otra informa conflicto, sin sobrescribir silenciosamente.	Integración concurrente.
E04	Perder la respuesta tras guardar y repetir la operación. Debe devolver el mismo resultado, con un solo aumento de versión.	Integración con fallo de red.
E05	Recibir versión 3 antes que 2 y reconectar otro cliente. Ningún cliente retrocede; todos recuperan la versión vigente.	Componente e integración.
E06	Editar y borrar a la vez, variando qué confirma primero. Una vez confirmado el borrado, ningún evento posterior restaura el mensaje.	Integración concurrente.
E07	Interrumpir antes del guardado y antes de publicar una edición guardada. No debe haber cambios parciales ni confirmaciones falsas; lo guardado se sincroniza al recuperarse.	Integración con fallos controlados.
E08	Cambiar texto válido por scripts o etiquetas. No debe ejecutar código en conversación, historial ni notificación.	API e interfaz.

Estos ocho escenarios forman parte de cada candidato a liberar. Las pruebas simultáneas coordinan el inicio de las operaciones para reproducir el conflicto; una espera fija no asegura que se pruebe esa condición.

14 Edición de mensajes y casos adicionales

Caso	Situación y resultado esperado	Validación
E09 P1	Editar justo antes, en y después de 15 min, alterando el reloj del cliente. Solo se acepta antes del límite, según el servidor.	Reloj controlado.
E10 P1	Enviar el mismo texto, vacío, espacios y límites Unicode. Igual contenido no crea versión; un rechazo tampoco modifica el mensaje.	Unidad y API.
E11 P1	Reutilizar la clave de edición con otro texto o mensaje. Debe informar conflicto sin modificar ninguno.	API e integración.
E12 P1	Editar un mensaje inexistente o borrado, con sesión vencida o sin membresía. Debe rechazar sin revelar datos ni restaurar contenido.	API de permisos.
E13 P1	Editar usando versiones soportadas distintas. El contrato debe permitir interpretar el texto y sus eventos sin corrupción.	Contratos y recorrido básico.
E14 P2	Cancelar, cerrar la aplicación y resolver un conflicto con teclado o lector de pantalla. No guarda una cancelación ni pierde texto local sin aviso.	Exploración por plataforma.
E15 P1	Buscar el texto editado y revisar vistas previas, notificaciones pendientes y auditoría. Las proyecciones propias convergen; se aclaran límites de servicios externos.	Integración.
E16 P2	Editar rápidamente con emojis e idiomas de derecha a izquierda. Las versiones avanzan y el texto sigue siendo legible.	Componente y exploración.

Si cambia el plazo de edición, E09 es obligatorio aunque sea P1. Si no cabe validar el riesgo de una historia, se reduce su alcance o se mantiene desactivada hasta completarla.

15 Ejecución del piloto en cada sprint

Control	Qué comprobar	Evidencia
X01 Compatibilidad	El candidato conserva los contratos de API y eventos de los clientes soportados.	Reporte con versiones de proveedor y consumidores.
X02 Despliegue y recuperación	Desplegar en pruebas, comprobar envío y edición, recuperar una versión compatible o desactivar el cambio y volver a comprobar.	Ejecución del ensayo y verificación de datos.
X03 Capacidad básica	Mantener 50 envíos/s durante 2 min, tras calentamiento, sin errores inesperados ni cola creciente. Comparar con el mismo ambiente.	Tiempos, errores y cola. No sustituye la prueba de millones diarios.
X04 Consistencia	Comparar IDs y versiones aceptadas con almacenamiento y clientes después de procesar los eventos.	Lista de registros comprobados; no solo conteos totales.

Los 16 escenarios P0 más X01 a X04 forman el conjunto de 20. Cada escenario puede requerir varias pruebas. Una prueba omitida o sin datos adecuados no cuenta como aprobada.

Selección según el cambio

En refinamiento identificamos el riesgo y los casos afectados. Si cambia autorización, priorizamos S03, E02 y los accesos relacionados; si cambia el manejo de reintentos, S02, S04 y E04. Antes de liberar se ejecuta el conjunto completo y los casos adicionales afectados. Las entregas pueden ocurrir cualquier día si cumplen los criterios.

Datos y trazabilidad

Usaré dos empresas de prueba, miembros y no miembros, varios clientes y datos independientes por ejecución. En el ambiente de pruebas simularé fallos de conexión, el paso del tiempo y la llegada de eventos en distinto orden. Así podré repetir situaciones como una edición fuera del plazo permitido o una confirmación de mensaje que llega tarde, sin depender de que ocurran por casualidad.

Cada ejecución guardará caso, historia, versión de código, ambiente, datos, resultado y evidencia. Por ejemplo: riesgo de duplicación → historia de envío → criterio de reintento → S02 → ejecución del pipeline → despliegue. El defecto conserva esa relación hasta comprobar su corrección.

En la revisión del sprint mostraré qué problemas se corrigieron, qué pruebas se incorporaron para prevenir o detectar fallos y qué sigue pendiente. En la retrospectiva elegiremos una mejora concreta que el equipo pueda completar durante la semana siguiente.